

www.BrainKart.com

Anna University

for Affiliated Engineering College - 2021 Regulation



AID (Artificial Intelligence & Data Science Engineering)

1st Semester ▶

2nd Semester ▶

3rd Semester ▶

4th Semester ▶

5th Semester ▶

6th Semester ▶

7th Semester ▶

8th Semester ▶

Click on Subject/Paper under Semester to enter.

1st Semester

[Professional English - I - HS3152](#)

[Matrices and Calculus - MA3151](#)

[Engineering Physics - PH3151](#)

[Engineering Chemistry - CY3151](#)

[Problem Solving and Python Programming - GE3151](#)

2nd Semester

[Professional English - II - HS3252](#)

[Statistics and Numerical Methods - MA3251](#)

[Engineering Graphics - GE3251](#)

[Physics for Information Science - PH3256](#)

[Basic Electrical and Electronics Engineering - BE3251](#)

[Data Structures Design - AD3251](#)

3rd Semester

[Discrete Mathematics - MA3354](#)

[Digital Principles and Computer Organization - CS3351](#)

[Database Design and Management - AD3391](#)

[Design and Analysis of Algorithms - AD3351](#)

[Data Exploration and Visualization - AD3301](#)

[Artificial Intelligence - AL3391](#)

4th Semester

[Environmental Sciences and Sustainability - GE3451](#)

[Probability and Statistics - MA3391](#)

[Operating Systems - AL3452](#)

[Machine Learning - AL3451](#)

[Fundamentals of Data Science and Analytics - AD3491](#)

[Computer Networks - CS3591](#)

5th Semester

[Deep Learning - AD3501](#)

[Data and Information Security - CW3551](#)

[Distributed Computing - CS3551](#)

[Big Data Analytics - CCS334](#)

[Elective 1](#)

[Elective 2](#)

6th Semester

[Embedded Systems and IoT - CS3691](#)

[Open Elective-1](#)

[Elective-3](#)

[Elective-4](#)

[Elective-5](#)

[Elective-6](#)

7th Semester

[Human Values and Ethics - GE3791](#)

[Open Elective 2](#)

[Open Elective 3](#)

[Open Elective 4](#)

[Management Elective](#)

8th Semester

[Project Work / Internship](#)



Anna University Notes

Therithal Info
Contains ads

3.7★

199 reviews

50K+

Downloads

3+

Rated for 3+ ©

Install



BrainKart: Learning, Study App

Therithal Info
Contains ads

4.5★

160 reviews

10K+

Downloads

3+

Rated for 3+ ©

Install

All Computer Engg Subjects - [B.E., M.E.,]

(Click on Subjects to enter)

<u>Programming in C</u>	<u>Computer Networks</u>	<u>Operating Systems</u>
<u>Programming and Data Structures I</u>	<u>Programming and Data Structure II</u>	<u>Problem Solving and Python Programming</u>
<u>Database Management Systems</u>	<u>Computer Architecture</u>	<u>Analog and Digital Communication</u>
<u>Design and Analysis of Algorithms</u>	<u>Microprocessors and Microcontrollers</u>	<u>Object Oriented Analysis and Design</u>
<u>Software Engineering</u>	<u>Discrete Mathematics</u>	<u>Internet Programming</u>
<u>Theory of Computation</u>	<u>Computer Graphics</u>	<u>Distributed Systems</u>
<u>Mobile Computing</u>	<u>Compiler Design</u>	<u>Digital Signal Processing</u>
<u>Artificial Intelligence</u>	<u>Software Testing</u>	<u>Grid and Cloud Computing</u>
<u>Data Ware Housing and Data Mining</u>	<u>Cryptography and Network Security</u>	<u>Resource Management Techniques</u>
<u>Service Oriented Architecture</u>	<u>Embedded and Real Time Systems</u>	<u>Multi - Core Architectures and Programming</u>
<u>Probability and Queueing Theory</u>	<u>Physics for Information Science</u>	<u>Transforms and Partial Differential Equations</u>
<u>Technical English</u>	<u>Engineering Physics</u>	<u>Engineering Chemistry</u>
<u>Engineering Graphics</u>	<u>Total Quality Management</u>	<u>Professional Ethics in Engineering</u>
<u>Basic Electrical and Electronics and Measurement Engineering</u>	<u>Problem Solving and Python Programming</u>	<u>Environmental Science and Engineering</u>





DHANALAKSHMI COLLEGE OF ENGINEERING, CHENNAI

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AD3501 DEEP LEARNING - NOTES

UNIT II CONVOLUTIONAL NEURAL NETWORKS

Convolution Operation -- Sparse Interactions -- Parameter Sharing -- Equivariance -- Pooling -- Convolution Variants: Strided -- Tiled -- Transposed and dilated convolutions; CNN Learning: Nonlinearity Functions -- Loss Functions -- Regularization -- Optimizers -- Gradient Computation.

1. Introduction to Convolutional Neural Networks:

A Convolutional Neural Network (CNN) is a type of Deep Learning neural network architecture commonly used in Computer Vision. Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data. In a regular Neural Network there are three types of layers:

- **Input Layers:** It's the layer in which we give input to our model. The number of neurons in this layer is equal to the total number of features in our data (number of pixels in the case of an image).
- **Hidden Layer:** The input from the Input layer is then feed into the hidden layer. There can be many hidden layers depending upon our model and data size. Each hidden layer can have different numbers of neurons which are generally greater than the number of features. The output from each layer is computed by matrix multiplication of output of the previous layer with learnable weights of that layer and then by the addition of learnable biases followed by activation function which makes the network nonlinear.
- **Output Layer:** The output from the hidden layer is then fed into a logistic function like sigmoid or softmax which converts the output of each class into the probability score of each class.

The data is fed into the model and output from each layer is obtained from the above step is called feed forward, we then calculate the error using an error function, some common error functions are cross-entropy, square loss error, etc. The error function measures how well the network is performing. After that, we back propagate into the model by calculating the derivatives. This step is called Back propagation which basically is used to minimize the loss.

1.1 Convolution Neural Network

Convolutional Neural Network (CNN) is the extended version of artificial neural networks_(ANN) which is predominantly used to extract the feature from the grid-like matrix dataset. For example visual datasets like images or videos where data patterns play an extensive role.

Around the 1980s, CNNs were developed and deployed for the first time. A CNN could only detect handwritten digits at the time. CNN was primarily used in various areas to read zip and pin codes etc. The most common aspect of any AI model is that it requires a massive amount of data to train. This was one of

the biggest problems that CNN faced at the time, and due to this, they were only used in the postal industry. Yann LeCun was the first to introduce convolutional neural networks.

Convolutional Neural Networks, commonly referred to as CNNs, are a specialized kind of neural network architecture that is designed to process data with a grid-like topology. This makes them particularly well-suited for dealing with spatial and temporal data, like images and videos that maintain a high degree of correlation between adjacent elements.

CNNs are similar to other neural networks, but they have an added layer of complexity due to the fact that they use a series of convolutional layers. Convolutional layers perform a mathematical operation called convolution, a sort of specialized matrix multiplication, on the input data. The convolution operation helps to preserve the spatial relationship between pixels by learning image features using small squares of input data. . The picture below represents a typical CNN architecture.

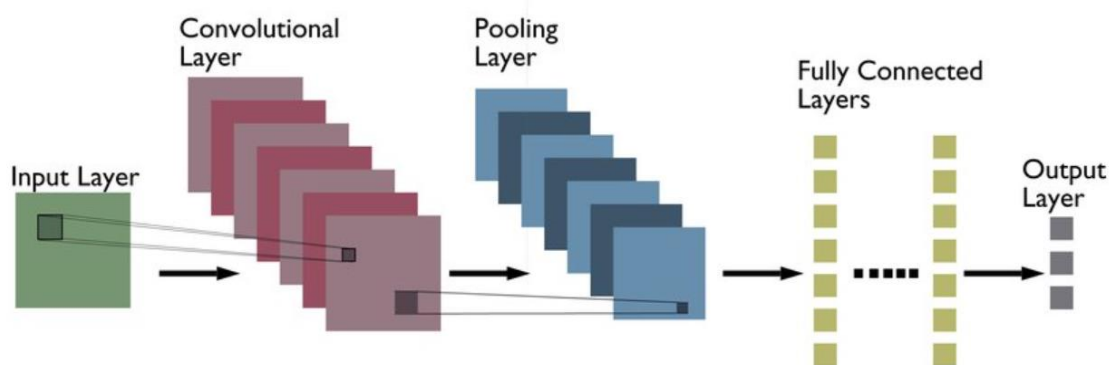


Fig. 1 Typical CNN architecture

The following are definitions of different layers shown in the above architecture:

- **Convolutional layers**

Convolutional layers operate by sliding a set of ‘filters’ or ‘kernels’ across the input data. Each filter is designed to detect a specific feature or pattern, such as edges, corners, or more complex shapes in the case of deeper layers. As these filters move across the image, they generate a map that signifies the areas where those features were found. The output of the convolutional layer is a feature map, which is a representation of the input image with the filters applied. Convolutional layers can be stacked to create more complex models, which can learn more intricate features from images. Simply speaking, convolutional layers are responsible for extracting features from the input images. These features might include edges, corners, textures, or more complex patterns.

- **Pooling layers**

Pooling layers follow the convolutional layers and are used to reduce the spatial dimension of the input, making it easier to process and requiring less memory. In the context of images, “spatial dimensions” refer to the width and height of the image. An image is made up of pixels, and you can think of it like a grid, with rows and columns of tiny squares (pixels). By reducing the spatial dimensions, pooling layers help reduce the number of parameters or weights in the network. This helps to combat over-fitting and help train the model in a fast manner. Max pooling helps in reducing computational complexity, owing to reduction in size of feature map, and making the model invariant to small translations. Without max pooling, the network would not gain the ability to recognize features irrespective of small shifts or

rotations. This would make the model less robust to variations in object positioning within the image, possibly affecting accuracy.

There are two main types of pooling: max pooling and average pooling. Max pooling takes the maximum value from each feature map. For example, if the pooling window size is 2×2 , it will pick the pixel with the highest value in that 2×2 region. Max pooling effectively captures the most prominent feature or characteristic within the pooling window. Average pooling calculates the average of all values within the pooling window. It provides a smooth, average feature representation.

- **Fully connected layers**

Fully-connected layers are one of the most basic types of layers in a convolutional neural network (CNN). As the name suggests, each neuron in a fully-connected layer is Fully connected- to every other neuron in the previous layer. Fully connected layers are typically used towards the end of a CNN- when the goal is to take the features learned by the convolutional and max pooling layers and use them to make predictions such as classifying the input to a label. For example, if we were using a CNN to classify images of animals, the final Fully connected layer might take the features learned by the previous layers and use them to classify an image as containing a dog, cat, bird, etc.

Fully connected layers take the high-dimensional output from the previous convolutional and pooling layers and flatten it into a one-dimensional vector. This allows the network to combine and integrate all the extracted features across the entire image, rather than considering localized features. It helps in understanding the global context of the image. The fully connected layers are responsible for mapping the integrated features to the desired output, such as class labels in classification tasks. They act as the final decision-making part of the network, determining what the extracted features mean in the context of the specific problem (e.g., recognizing a cat or a dog).

The combination of Convolution layer followed by max-pooling layer and then similar sets creates a hierarchy of features. The first layer detects simple patterns, and subsequent layers build on those to detect more complex patterns.

CNNs are often used for image recognition and classification tasks. For example, CNNs can be used to identify objects in an image or to classify an image as being a cat or a dog. CNNs can also be used for more complex tasks, such as generating descriptions of an image or identifying the points of interest in an image. Beyond image data, CNNs can also handle time-series data, such as audio data or even text data, although other types of networks like Recurrent Neural Networks (RNNs) or transformers are often preferred for these scenarios. CNNs are a powerful tool for deep learning, and they have been used to achieve state-of-the-art results in many different applications.

1.2 CNN architecture

Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers.

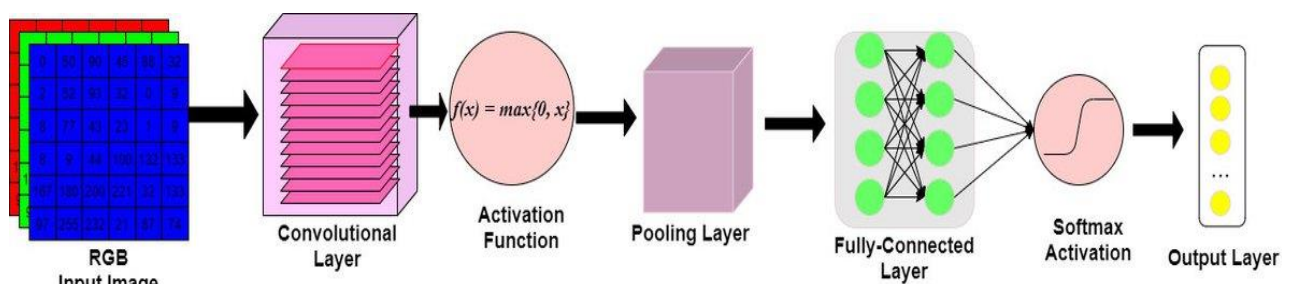


Fig.2 Simple CNN architecture

The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through back propagation and gradient descent as detailed in Fig. 3.

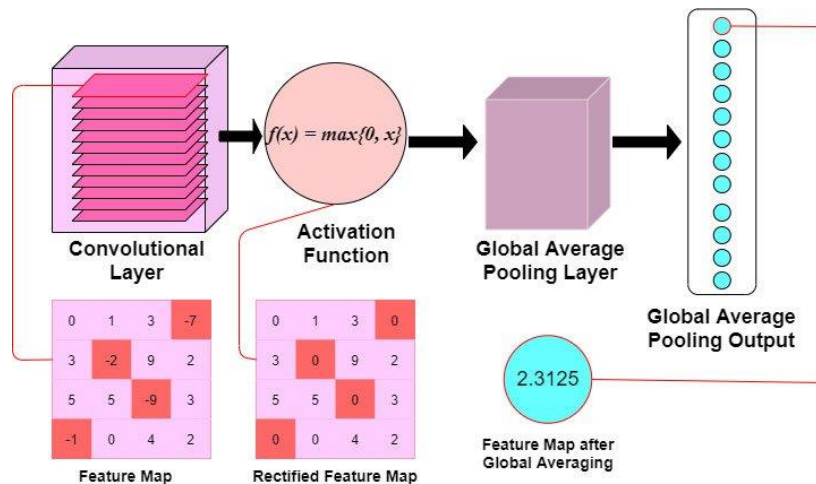


Fig. 3 Functions of CNN Layers

1.2.1 Different types of CNN Architectures

The following is a list of different types of CNN architectures:

LeNet: LeNet is the first CNN architecture. It was developed in 1998 by Yann LeCun, Corinna Cortes, and Christopher Burges for handwritten digit recognition problems. LeNet was one of the first successful CNNs and is often considered the “Hello World” of deep learning. It is one of the earliest and most widely-used CNN architectures and has been successfully applied to tasks such as handwritten digit recognition. The LeNet architecture consists of multiple convolutional and pooling layers, followed by a fully-connected layer. The model has five convolution layers followed by two fully connected layers. LeNet was the beginning of CNNs in deep learning for computer vision problems. However, LeNet could not train well due to the vanishing gradients problem. To solve this issue, a shortcut connection layer known as max-pooling is used between convolutional layers to reduce the spatial size of images which helps prevent overfitting and allows CNNs to train more effectively. The diagram below represents LeNet-5 architecture.

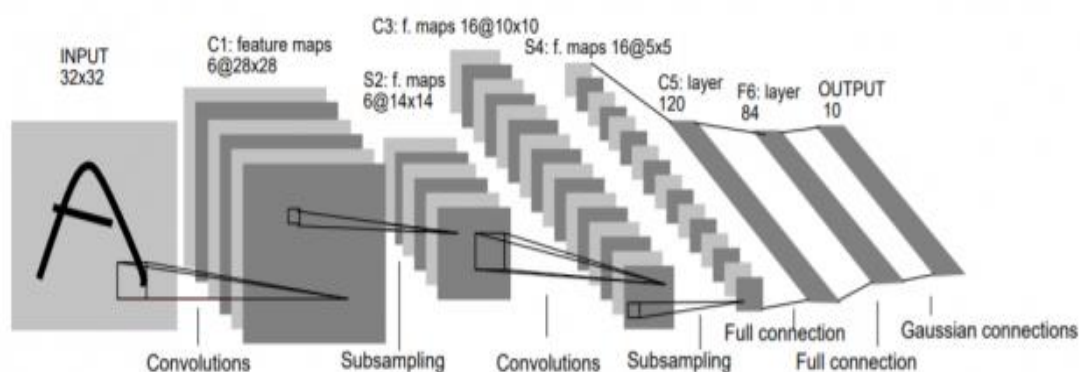


Fig. 4 LeNet Architecture

The LeNet CNN is a simple yet powerful model that has been used for various tasks such as handwritten digit recognition, traffic sign recognition, and face detection. Although LeNet was developed more than 20 years ago, its architecture is still relevant today and continues to be used.

AlexNet: AlexNet is the deep learning architecture that popularized CNN. It was developed by Alex Krizhevsky, Ilya Sutskever, and Geoff Hinton. AlexNet network had a very similar architecture to LeNet, but was deeper, bigger, and featured Convolutional Layers stacked on top of each other. AlexNet was the first large-scale CNN and was used to win the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012. The AlexNet architecture was designed to be used with large-scale image datasets and it achieved state-of-the-art results at the time of its publication. AlexNet is composed of 5 convolutional layers with a combination of max-pooling layers, 3 fully connected layers, and 2 dropout layers. The activation function used in all layers is Relu. The activation function used in the output layer is Softmax. The total number of parameters in this architecture is around 60 million.

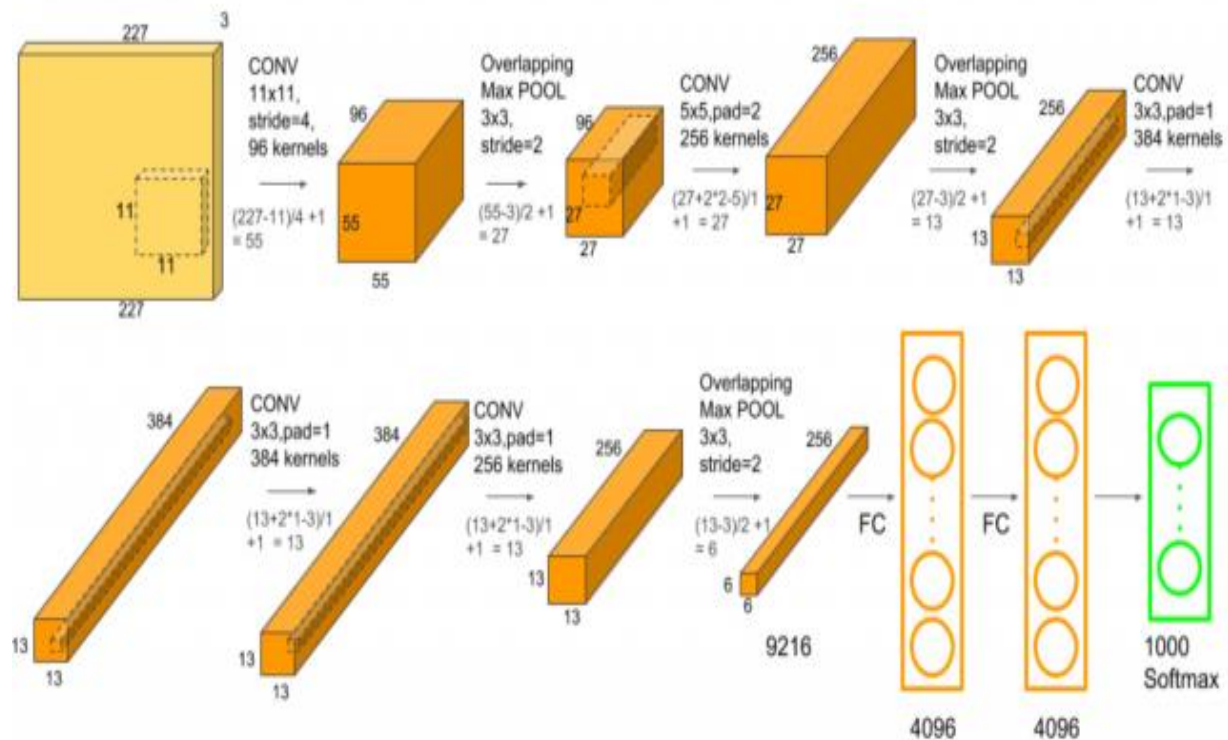


Fig. 5 AlexNet Architecture

ZF Net: ZFnet is the CNN architecture that uses a combination of fully-connected layers and CNNs. ZF Net was developed by Matthew Zeiler and Rob Fergus. It was the ILSVRC 2013 winner. The network has relatively fewer parameters than AlexNet, but still outperforms it on ILSVRC 2012 classification task by achieving top accuracy with only 1000 images per class. It was an improvement on AlexNet by tweaking the architecture hyperparameters, in particular by expanding the size of the middle convolutional layers and making the stride and filter size on the first layer smaller. It is based on the Zeiler and Fergus model, which was trained on the ImageNet dataset. ZF Net CNN architecture consists of a total of seven layers: Convolutional layer, max-pooling layer (downscaling), concatenation layer, convolutional layer with linear activation function, and stride one, dropout for regularization purposes applied before the fully connected output. This CNN model is computationally more efficient than AlexNet by introducing an approximate inference stage through deconvolutional layers in the middle of CNNs.

GoogLeNet: GoogLeNet is the CNN architecture used by Google to win ILSVRC 2014 classification task. It was developed by Jeff Dean, Christian Szegedy, Alexandro Szegedy et al.. It has been shown to have a notably reduced error rate in comparison with previous winners AlexNet (Ilsvrc 2012 winner) and ZF-Net (Ilsvrc 2013 winner). In terms of error rate, the error is significantly lesser than VGG (2014 runner up). It achieves deeper architecture by employing a number of distinct techniques, including 1x1 convolution and

global average pooling. GoogLeNet CNN architecture is computationally expensive. To reduce the parameters that must be learned, it uses heavy unpooling layers on top of CNNs to remove spatial redundancy during training and also features shortcut connections between the first two convolutional layers before adding new filters in later CNN layers. Real-world applications/examples of GoogLeNet CNN architecture include Street View House Number (SVHN) digit recognition task, which is often used as a proxy for roadside object detection. Below is the simplified block diagram representing GoogLeNet CNN architecture:

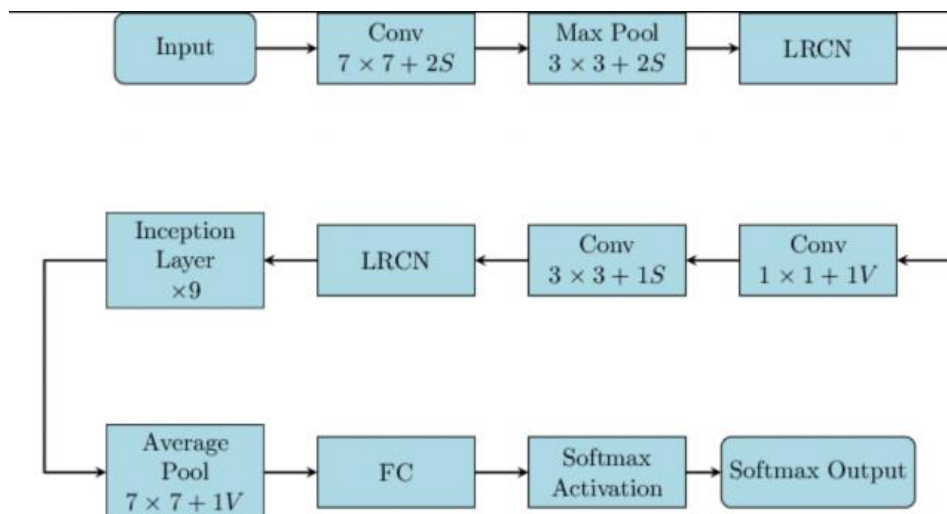


Fig. 6 GoogLeNet Architecture

VGGNet: VGGNet is the CNN architecture that was developed by Karen Simonyan, Andrew Zisserman et al. at Oxford University. VGGNet is a 16-layer CNN with up to 95 million parameters and trained on over one billion images (1000 classes). It can take large input images of 224 x 224-pixel size for which it has 4096 convolutional features. CNNs with such large filters are expensive to train and require a lot of data, which is the main reason why CNN architectures like GoogLeNet (AlexNet architecture) work better than VGGNet for most image classification tasks where input images have a size between 100 x 100-pixel and 350 x 350 pixels. Real-world applications/examples of VGGNet CNN architecture include the ILSVRC 2014 classification task, which was also won by GoogLeNet CNN architecture. The VGG CNN model is computationally efficient and serves as a strong baseline for many applications in computer vision due to its applicability for numerous tasks including object detection. Its deep feature representations are used across multiple neural network architectures like YOLO, SSD, etc. The diagram below represents the standard VGG16 network architecture diagram:

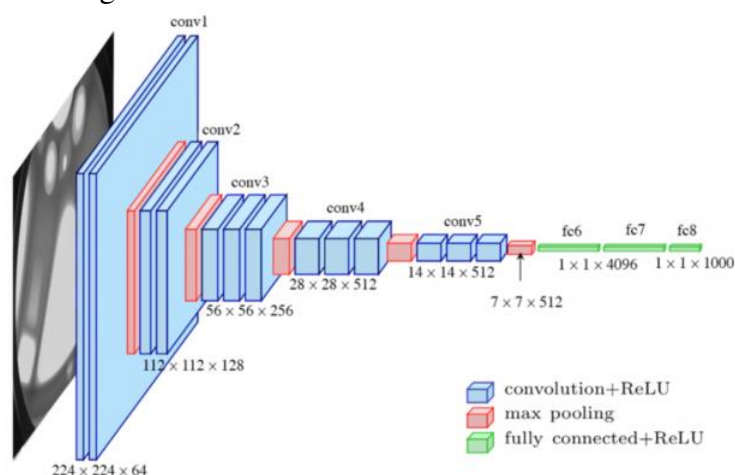


Fig. 7 VGGNet Architecture

ResNet: ResNet is the CNN architecture that was developed by Kaiming He et al. to win the ILSVRC 2015 classification task with a top-five error of only 15.43%. The network has 152 layers and over one million parameters, which is considered deep even for CNNs because it would have taken more than 40 days on 32 GPUs to train the network on the ILSVRC 2015 dataset. CNNs are mostly used for image classification tasks with 1000 classes, but ResNet proves that CNNs can also be used successfully to solve natural language processing problems like sentence completion or machine comprehension, where it was used by the Microsoft Research Asia team in 2016 and 2017 respectively. Real-life applications/examples of ResNet CNN architecture include Microsoft's machine comprehension system, which has used CNNs to generate the answers for more than 100k questions in over 20 categories. The CNN architecture ResNet is computationally efficient and can be scaled up or down to match the computational power of GPUs.

MobileNets: MobileNets are CNNs that can be fit on a mobile device to classify images or detect objects with low latency. MobileNets have been developed by Andrew G Trillion et al.. They are usually very small CNN architectures, which makes them easy to run in real-time using embedded devices like smartphones and drones. The architecture is also flexible so it has been tested on CNNs with 100-300 layers and it still works better than other architectures like VGGNet. Real-life examples of MobileNets CNN architecture include CNNs that is built into Android phones to run Google's Mobile Vision API, which can automatically identify labels of popular objects in images.

GoogLeNet_DeepDream: GoogLeNet_DeepDream is a deep dream CNN architecture that was developed by Alexander Mordvintsev, Christopher Olah, et al.. It uses the Inception network to generate images based on CNN features. The architecture is often used with the ImageNet dataset to generate psychedelic images or create abstract artworks using human imagination at the ICLR 2017 workshop by David Ha, et al.

To summarize the different types of CNN architectures described above in an easy to remember form, you can use the following:

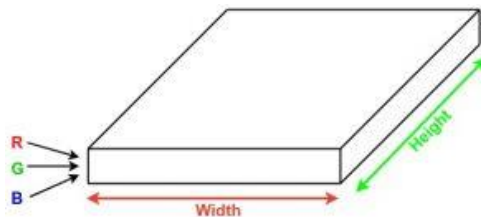
Table 1. Different Types of CNN Architectures

Architecture	Year	Key Features	Use Case
LeNet	1998	First successful applications of CNNs, 5 layers (alternating between convolutional and pooling), Used tanh/sigmoid activation functions	Recognizing handwritten and machine-printed characters
AlexNet	2012	Deeper and wider than LeNet, Used ReLU activation function, Implemented dropout layers, Used GPUs for training	Large-scale image recognition tasks
ZFNet	2013	Similar architecture to AlexNet, but with different filter sizes and numbers of filters, Visualization techniques for understanding the network	ImageNet classification
VGGNet	2014	Deeper networks with smaller filters (3×3), All convolutional layers have the same depth, Multiple configurations (VGG16, VGG19)	Large-scale image recognition
ResNet	2015	Introduced “skip connections” or “shortcuts” to enable training of deeper networks, Multiple configurations (ResNet-50, ResNet-101, ResNet-152)	Large-scale image recognition, won 1st place in the ILSVRC 2015
GoogLeNet	2014	Introduced Inception module, which allows for more efficient computation and deeper networks, multiple versions (Inception v1, v2, v3, v4)	Large-scale image recognition, won 1st place in the ILSVRC 2014

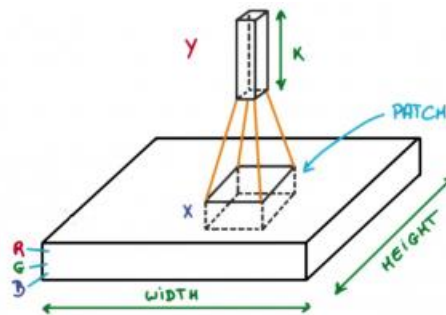
Architecture	Year	Key Features	Use Case
MobileNets	2017	Designed for mobile and embedded vision applications, Uses depthwise separable convolutions to reduce the model size and complexity	Mobile and embedded vision applications, real-time object detection
LeNet	1998	First successful applications of CNNs, 5 layers (alternating between convolutional and pooling), Used tanh/sigmoid activation functions	Recognizing handwritten and machine-printed characters

1.3 Working of Convolutional Layers

Convolution Neural Networks or convnets are neural networks that share their parameters. Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e the channel as images generally have red, green, and blue channels).



Now imagine taking a small patch of this image and running a small neural network, called a filter or kernel on it, with say, K outputs and representing them vertically. Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths. Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called **Convolution**. If the patch size is the same as that of the image it will be a regular neural network. Because of this small patch, we have fewer weights.



Now let's talk about a bit of mathematics that is involved in the whole convolution process.

Convolution layers consist of a set of learnable filters (or kernels) having small widths and heights and the same depth as that of input volume (3 if the input layer is image input).

For example, if we have to run convolution on an image with dimensions $34 \times 34 \times 3$. The possible size of filters can be $a \times a \times 3$, where 'a' can be anything like 3, 5, or 7 but smaller as compared to the image dimension.

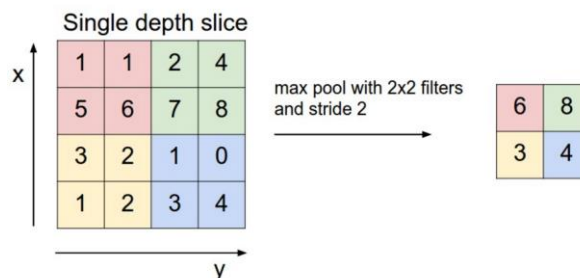
During the forward pass, we slide each filter across the whole input volume step by step where each step is called **stride** (which can have a value of 2, 3, or even 4 for high-dimensional images) and compute the dot product between the kernel weights and patch from input volume.

As we slide our filters we'll get a 2-D output for each filter and we'll stack them together as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

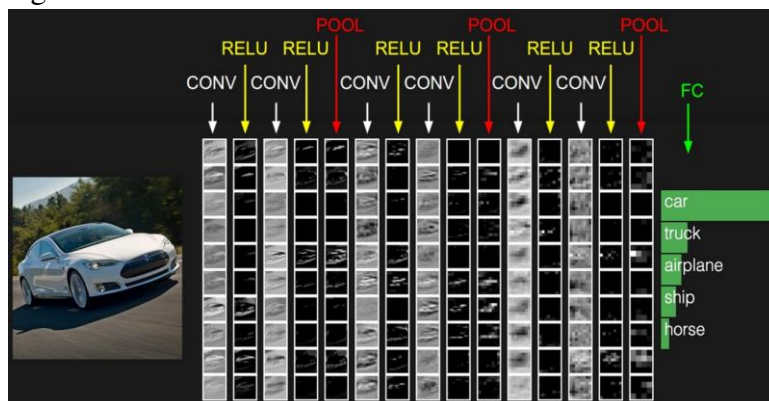
1.3.1 Layers used to build ConvNets

A complete Convolution Neural Networks architecture is also known as convNets. A convNets is a sequence of layers, and every layer transforms one volume to another through a differentiable function. Let's take an example by running a convNets on of image of dimension 32 x 32 x 3.

- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
- **Convolutional Layers:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2x2, 3x3, or 5x5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred ad feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension 32 x 32 x 12.
- **Activation Layer:** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are **RELU**, **Tanh**, **Leaky RELU**, etc. The volume remains unchanged hence output volume will have dimensions 32 x 32 x 12.
- **Pooling layer:** This layer is periodically inserted in the convnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents over-fitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.



- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.



- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

1.4 Advantages of Convolutional Neural Networks (CNNs)

- Good at detecting patterns and features in images, videos, and audio signals
- Robust to translation, rotation, and scaling invariance
- End-to-end training, no need for manual feature extraction
- Can handle large amounts of data and achieve high accuracy

1.5 Disadvantages of Convolutional Neural Networks (CNNs)

- Computationally expensive to train and require a lot of memory
- Can be prone to over-fitting if not enough data or proper regularization is used
- Requires large amounts of labeled data
- Interpretability is limited, it's hard to understand what the network has learned

1.6 Applications of CNN

Here are some of the common applications of convolutional neural networks:

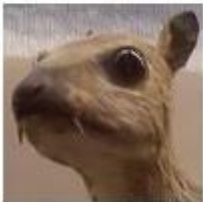
- Semantic segmentation: CNNs can classify every pixel in an image into different classes, for e.g. - different types of vegetation in satellite images.
- Object detection: CNNs can detect objects within an image, for e.g. - identifying the location & the type of vehicle on the road.
- Image classification: CNNs can classify images into different categories, for e.g. identifying objects in a photograph.
- Image captioning: CNNs can generate natural language descriptions of images, for e.g. - describing the objects in a photograph.
- Face recognition - CNNs can recognize & verify the identity of different individuals in images, such as finding people's faces in security footages.
- Medical image analysis - CNNs can identify tumors in medical scans, or in detecting abnormalities in X rays.
- Video analysis - CNNs can detect the movement of objects across frames.
- Autonomous vehicles - CNNs can identify & track objects - such as pedestrians & other vehicles.

2. Convolution Operation:

A convolutional neural network, or ConvNet, is just a neural network that uses convolution. To understand the principle, we are going to work with a 2-dimensional convolution first.

Convolution is a mathematical operation that allows the merging of two sets of information. Convolution between two functions in mathematics produces a third function expressing how the shape of one function is modified by other. In the case of CNN, convolution is applied to the input data to filter the information and produce a feature map.

This filter is also called a kernel, or feature detector, and its dimensions can be, for example, 3x3. A kernel is a small 2D matrix whose contents are based upon the operations to be performed. A kernel maps on the input image by simple matrix multiplication and addition, the output obtained is of lower dimensions and therefore easier to work with.

Original	Gaussian Blur	Sharpen	Edge Detection
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

Above is an example of a kernel for applying Gaussian blur (to smoothen the image before processing), Sharpen image (enhance the depth of edges) and edge detection. To perform convolution, the kernel goes over the input image, doing matrix multiplication element after element. The result for each receptive field (the area where convolution takes place) is written down in the feature map.

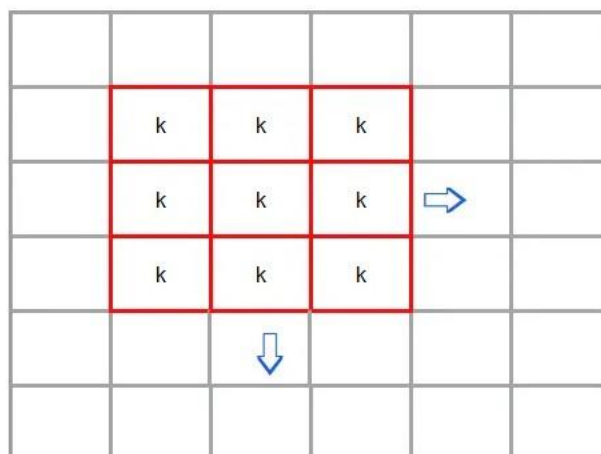
Input

0x1	1x0	1x1	0	0
0x1	1x1	0x1	1	0
1x0	1x0	0x1	1	1
0	0	1	1	0
0	1	1	0	0

Filter / Kernel

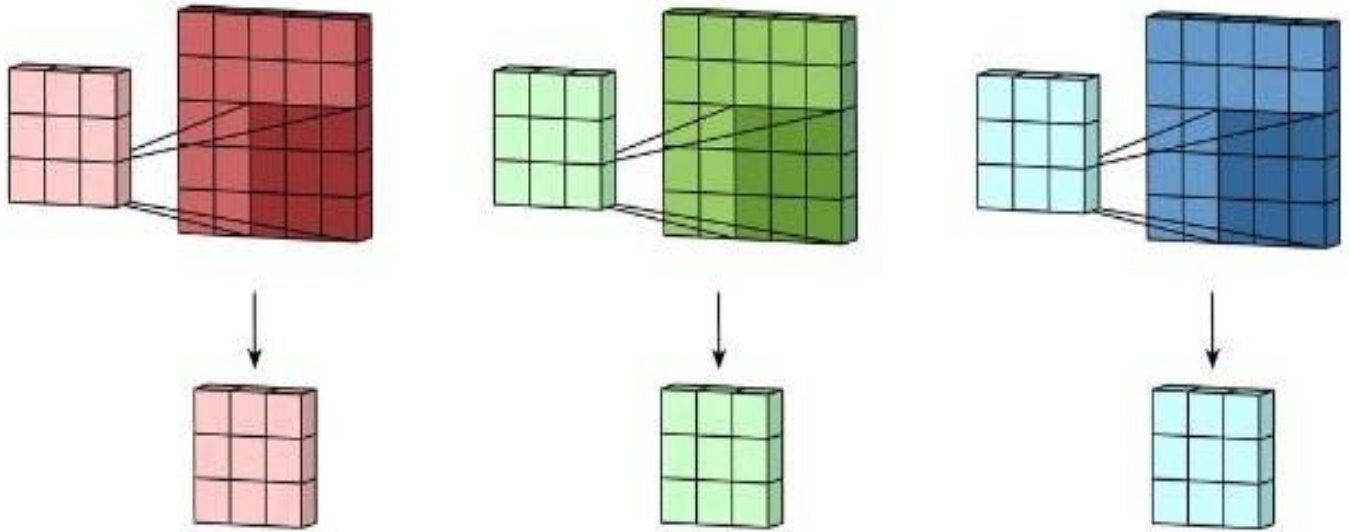
2		

The shape of a kernel is heavily dependent on the input shape of the image and architecture of the entire network, mostly the size of kernels is (**MxM**) i.e., a square matrix. The movement of a kernel is always from left to right and top to bottom.



Stride defines by what step does to kernel move, for example stride of 1 makes kernel slide by one

row/column at a time and stride of 2 moves kernel by 2 rows/columns. We continue sliding the filter until the feature map is complete.



For input images with 3 or more channels such as RGB a filter is applied. Filters are one dimension higher than kernels and can be seen as multiple kernels stacked on each other where every kernel is for a particular channel. Therefore for an RGB image of (32x32) we have a filter of the shape say (5x5x3).

Input Matrix				Kernel			Result	
45	12	5	17	0	-1	0	-45	12
22	10	35	6	-1	5	-1	22	10
88	26	51	19	0	-1	0		
9	77	42	3					

45	12	5	17
22	10	35	6
88	26	51	19
9	77	42	3

0	-1	0
-1	5	-1
0	-1	0

-45	103
22	10

45	12	5	17
22	10	35	6
88	26	51	19
9	77	42	3

0	-1	0
-1	5	-1
0	-1	0

-45	103
96	10

45	12	5	17
22	10	35	6
88	26	51	19
9	77	42	3

0	-1	0
-1	5	-1
0	-1	0

-45	103
-176	133

Here the input matrix has shape 4x4x1 and the kernel is of size 3x3 since the shape of input is larger than the kernel, we are able to implement a sliding window protocol and apply the kernel over entire input. First entry in the convoluted result is calculated as:

$$45*0+12*(-1)+5*0+22*(-1)+10*5+35*(-1)+88*0+26*(-1)+51*0 = -45$$

We continue sliding the filter until the feature map is complete.

2.1 Sliding window protocol:

1. The kernel gets into position at the top-left corner of the input matrix.
2. Then it starts moving left to right, calculating the dot product and saving it to a new matrix until it has reached the last column.
3. Next, kernel resets its position at first column but now it slides one row to the bottom. Thus following the fashion left-right and top-bottom.
4. Steps 2 and 3 are repeated till the entire input has been processed.

For a 3D input matrix the movement of the kernel will be from front to back, left to right and top to bottom.

2.2 Sparse Interactions (Connectivity)

Convolutional neural networks are more efficient than simple neural networks — in applications where they apply, because they significantly reduce the number of parameters which reduces the required memory of the network and improves its statistical efficiency. They exploit **feature locality**. They try to find patterns in the input data. They stack them to make abstract concepts by their convolution layers. A **Convolution layer** defines a window or filter or kernel by which they examine a subset of the data, and subsequently scans the data looking through this window. We can parameterize the window to look for specific features (e.g. edges within an image). The output they produce focuses solely on the regions of the data which exhibited the feature it was searching for. This is what we call **sparse connectivity** or **sparse interactions** or **sparse weights**. Actually it limits the activated connections at each layer. In the example below an 5x5 input with a 2x2 filter produces a reduced 4x4 output. The first element of feature map is calculated by the convolution of the input area with the filter i.e.

Input Image									
1 × 0	2 × 1	1	3	1					
2 × 1	1 × 2	1	2	1					
2	1	1	1	3					
0	1	2	2	3					
4	1	3	1	0					

					Feature Map				
					6				

$$1 \times 0 + 2 \times 1 + 2 \times 1 + 1 \times 2 = 6$$

In practice, we don't explicitly define the filters that our convolutional layer will use; we instead parameterize the filters and let the network learn the best filters to use during training. We do, however, define how many filters, we'll use at each layer— a hyperparameter which is called the **depth** of the output volume.

Another hyperparameter is the **stride** that defines how much we slide the filter over the data. For example if stride is 1 then we move the window by 1 pixel at a time over the image, when our input is an image. When we use larger values of stride 2 or 3 we allow jumping 2 or pixels at a time. This reduces significantly the output size.

The last hyperparameter is the size of **zero-padding**, when sometimes is convenient to pad the input volume with zeros around the border.

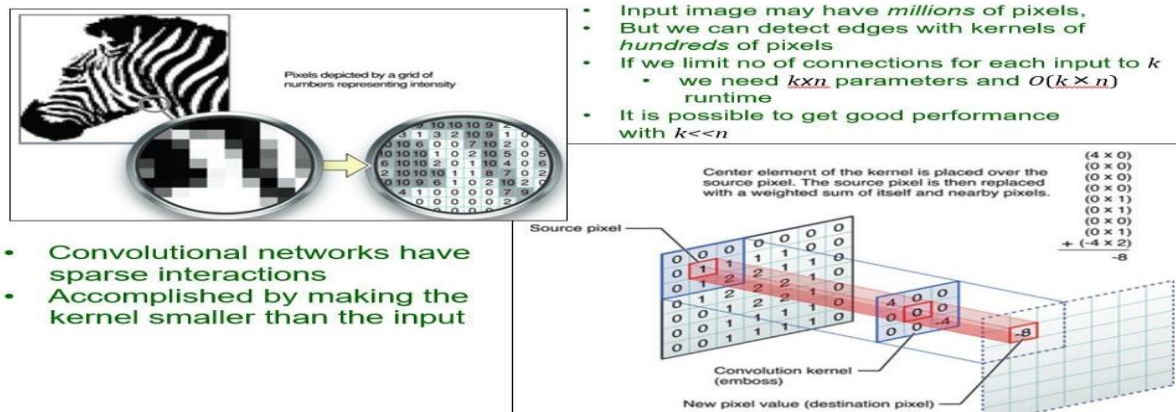
So now we can compute the spatial size of the output volume as a function of the input volume size (W), the receptive field size of the Conv Layer neurons (F), the stride with which they are applied (S), and the

amount of zero padding used (P) on the border. The formula for calculating how many neurons “fit” is given by

$$\frac{W - F + 2P}{S} + 1$$

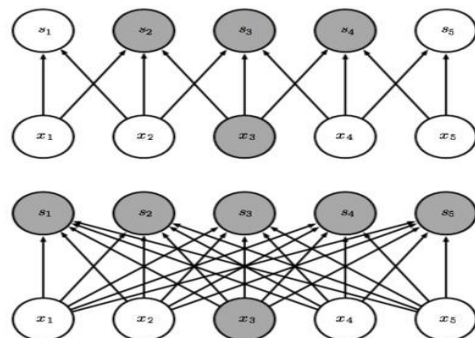
In our previous example for the 5x5 input (W=5) and the 2x2 filter (F=2) with stride 1(S=1) and pad 0 (P=0) we would get a 4x4x (number of filters) output for each network node.

Trivial neural network layers use matrix multiplication by a matrix of parameters describing the interaction between the input and output unit. This means that every output unit interacts with every input unit. However, convolution neural networks have **sparse interaction**. This is achieved by making kernel smaller than the input e.g., an image can have millions or thousands of pixels, but while processing it using kernel we can detect meaningful information that is of tens or hundreds of pixels. This means that we need to store fewer parameters that not only reduces the memory requirement of the model but also improves the statistical efficiency of the model.



Sparse Connectivity, viewed from below

- Highlight one input x_3 and output units s affected by it
- Top: when s is formed by convolution with a kernel of width 3, only three outputs are affected by x_3
- Bottom: when s is formed by matrix multiplication connectivity is no longer sparse
 - So all outputs are affected by x_3

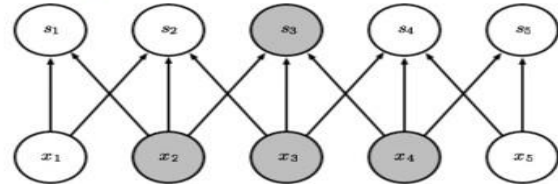


7

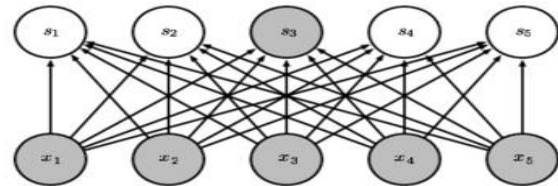
Sparse Connectivity, viewed from above

- Highlight one output s_3 and inputs x that affect this unit
 - These units are known as the receptive field of s_3

When s_3 is formed by convolution with a kernel of width 3

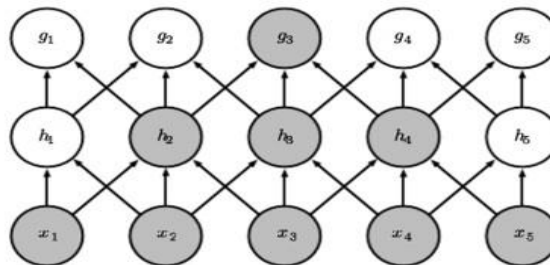


When s_3 is formed by matrix multiplication



Keeping up performance with reduced connections

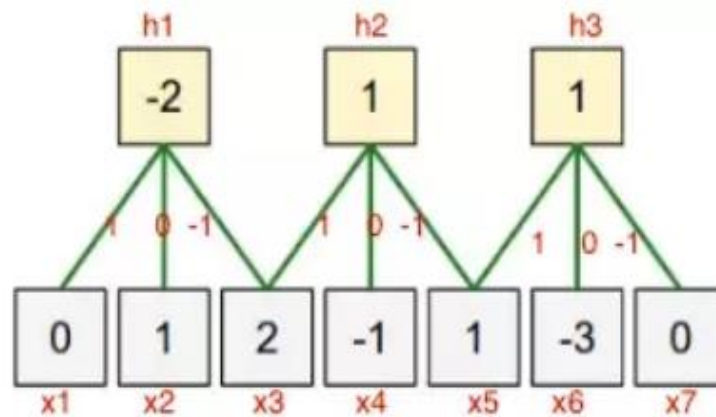
- It is possible to obtain good performance while keeping k several magnitudes lower than m
- In a deep neural network, units in deeper layers may indirectly interact with a larger portion of the input
 - Receptive Field in Deeper layers is larger than the receptive field of units in shallow layers



- This allows the network to efficiently describe complicated interactions between many variables from simple building blocks that only describe sparse interactions

2.3 Parameter (Weight) Sharing

If computing one feature at a spatial point (x_1, y_1) is useful then it should also be useful at some other spatial point say (x_2, y_2) . It means that for a single two-dimensional slice i.e., for creating one activation map, neurons are constrained to use the same set of weights. In a traditional neural network, each element of the weight matrix is used once and then never revisited, while convolution network has **shared parameters** i.e., for getting output, weights applied to one input are the same as the weight applied elsewhere. Parameter sharing is used in the convolutional layers to reduce the number of parameters in the network. For example in the first convolutional layer let's say we have an output of $15 \times 15 \times 4$ where 15 is the size of the output and 4 the number of filters used in this layer. For each output node in that layer we have the same filter, thus reducing dramatically the storage requirements of the model to the size of the filter.



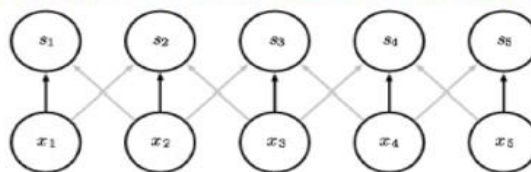
The same filter (weights) (1, 0, -1) are used for that layer.

Parameter Sharing

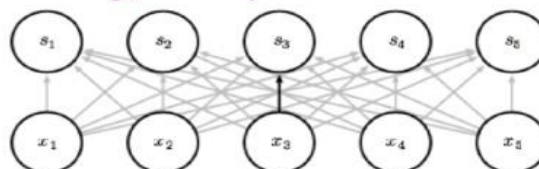
- *Parameter sharing* refers to using the same parameter for more than one function in a model
- In a traditional neural net each element of the weight matrix is used exactly once when computing the output of a layer
 - It is multiplied by one element of the input and never revisited
- Parameter sharing is synonymous with *tied weights*
 - Value of the weight applied to one input is tied to a weight applied elsewhere
- In a Convolutional net, each member of the kernel is used in every position of the input (except at the boundary—subject to design decisions)

How parameter sharing works

- Black arrows: connections that use a particular parameter
 1. Convolutional model: Black arrows indicate uses of the central element of a 3-element kernel



2. Fully connected model: Single black arrow indicates use of the central element of the weight matrix
 - Model has no parameter sharing, so the parameter is used only once



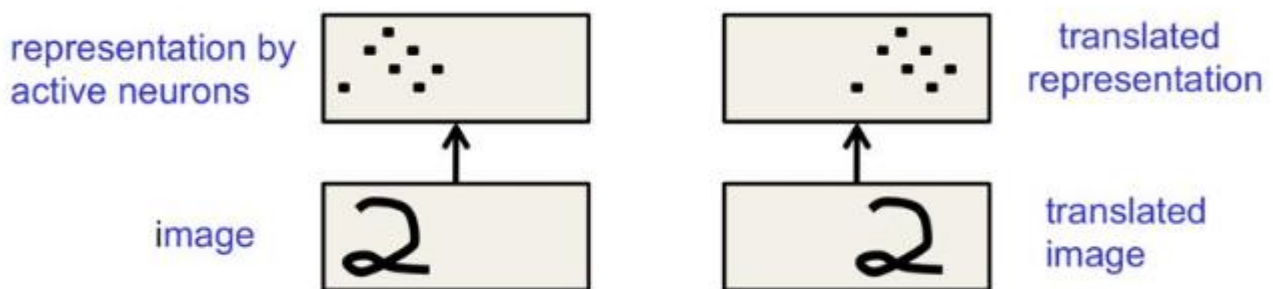
Efficiency of Parameter Sharing

- Parameter sharing by convolution operation means that rather than learning a separate set of parameters for every location, we learn only one set
- This does not affect runtime of forward propagation— which is still $O(k \times n)$
- But further reduces the storage requirements to k parameters
 - k is orders of magnitude less than m
 - Since m and n are roughly the same size k is much smaller than $m \times n$

2.4 Equivariant Representations

Equivariant means varying in the similar or equivalent proportion. Due to parameter sharing, the layers of convolution neural network will have a property of **equivariance to translation**. It says that if we changed the input in a way, the output will also get changed in the same way.

Equivariant to translation means that a translation of input features results in an equivalent translation of outputs. It makes the CNN understand the rotation or proportion change. The equivariance allows the network to generalize edge, texture, shape, detection in different locations.



Equivariance of Convolution to Translation

- The particular form of parameter sharing leads to equivariance to translation
- Equivariant means that if the input changes, the output changes in the same way
- A function $f(x)$ is equivariant to a function g if $f(g(x)) = g(f(x))$
- If g is a function that translates the input, i.e., that shifts it, then the convolution function is equivariant to g
 - $I(x,y)$ is image brightness at point (x,y)
 - $I' = g(I)$ is image function with $I'(x,y) = I(x-1,y)$, i.e., shifts every pixel of I one unit to the right
 - If we apply g to I and then apply convolution, the output will be the same as if we applied convolution to I' , then applied transformation g to the output

In some cases, we may not wish to share parameters across entire image

- If image is cropped to be centered on a face, we may want different features from different parts of the face
- Part of the network processing the top of the face looks for eyebrows
- Part of the network processing the bottom of the face looks for the chin
- Certain image operations such as scale and rotation are not equivariant to convolution
- Other mechanisms are needed for such transformations

2.5 Pooling

The pooling operation involves sliding a two-dimensional filter over each channel of feature map and summarizing the features lying within the region covered by the filter.

For a feature map having dimensions $n_h \times n_w \times n_c$, the dimensions of output obtained after a pooling layer is

$$(n_h - f + 1) / s \times (n_w - f + 1) / s \times n_c$$

where,

- n_h - height of feature map
- n_w - width of feature map
- n_c - number of channels in the feature map
- f - size of filter
- s - stride length

A common CNN model architecture is to have a number of convolution and pooling layers stacked one after the other.

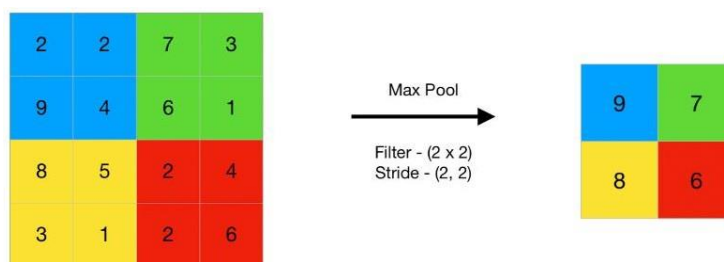
Pooling layers are used to reduce the dimensions of the feature maps. Thus, it reduces the number of parameters to learn and the amount of computation performed in the network.

The pooling layer summarizes the features present in a region of the feature map generated by a convolution layer. So, further operations are performed on summarized features instead of precisely positioned features generated by the convolution layer. This makes the model more robust to variations in the position of the features in the input image.

2.5.1 Types of Pooling Layers:

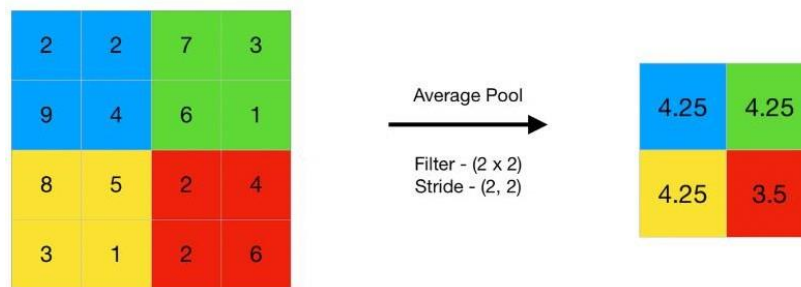
- **Max Pooling**

Max pooling is a pooling operation that selects the maximum element from the region of the feature map covered by the filter. Thus, the output after max-pooling layer would be a feature map containing the most prominent features of the previous feature map.



- **Average Pooling**

Average pooling computes the average of the elements present in the region of feature map covered by the filter. Thus, while max pooling gives the most prominent feature in a particular patch of the feature map, average pooling gives the average of features present in a patch.



- **Global Pooling**

Global pooling reduces each channel in the feature map to a single value. Thus, an $n_h \times n_w \times n_c$ feature map is reduced to $1 \times 1 \times n_c$ feature map. This is equivalent to using a filter of dimensions $n_h \times n_w$ i.e. the dimensions of the feature map. Further, it can be either global max pooling or global average pooling.

- **Global Average Pooling**

Considering a tensor of shape $h \times w \times n$, the output of the Global Average Pooling layer is a single value across $h \times w$ that summarizes the presence of the feature. Instead of downsizing the patches of the input feature map, the Global Average Pooling layer downsizes the whole $h \times w$ into 1 value by taking the average.

- **Global Max Pooling**

With the tensor of shape $h \times w \times n$, the output of the Global Max Pooling layer is a single value across $h \times w$ that summarizes the presence of a feature. Instead of downsizing the patches of the input feature map, the Global Max Pooling layer downsizes the whole $h \times w$ into 1 value by taking the maximum.

In convolutional neural networks (CNNs), the pooling layer is a common type of layer that is typically added after convolutional layers. The pooling layer is used to reduce the spatial dimensions (i.e., the width and height) of the feature maps, while preserving the depth (i.e., the number of channels).

- The pooling layer works by dividing the input feature map into a set of non-overlapping regions, called pooling regions. Each pooling region is then transformed into a single output value, which represents the presence of a particular feature in that region. The most common types of pooling operations are max pooling and average pooling.
- In max pooling, the output value for each pooling region is simply the maximum value of the input values within that region. This has the effect of preserving the most salient features in each pooling region, while discarding less relevant information. Max pooling is often used in CNNs for object recognition tasks, as it helps to identify the most distinctive features of an object, such as its edges and corners.
- In average pooling, the output value for each pooling region is the average of the input values within that region. This has the effect of preserving more information than max pooling, but may also dilute the most salient features. Average pooling is often used in CNNs for tasks such as image segmentation and object detection, where a more fine-grained representation of the input is required.

Pooling layers are typically used in conjunction with convolutional layers in a CNN, with each pooling layer reducing the spatial dimensions of the feature maps, while the convolutional layers extract increasingly complex features from the input. The resulting feature maps are then passed to a fully connected layer, which performs the final classification or regression task.

2.5.2 Advantages of Pooling Layer

Dimensionality reduction: The main advantage of pooling layers is that they help in reducing the spatial dimensions of the feature maps. This reduces the computational cost and also helps in avoiding over-fitting by reducing the number of parameters in the model.

Translation invariance: Pooling layers are also useful in achieving translation invariance in the feature maps. This means that the position of an object in the image does not affect the classification result, as the same features are detected regardless of the position of the object.

Feature selection: Pooling layers can also help in selecting the most important features from the input, as max pooling selects the most salient features and average pooling preserves more information.

2.5.3 Disadvantages of Pooling Layer

- **Information loss:** One of the main disadvantages of pooling layers is that they discard some information from the input feature maps, which can be important for the final classification or regression task.
- **Over-smoothing:** Pooling layers can also cause over-smoothing of the feature maps, which can result in the loss of some fine-grained details that are important for the final classification or regression task.
- **Hyperparameter tuning:** Pooling layers also introduce hyperparameters such as the size of the pooling regions and the stride, which need to be tuned in order to achieve optimal performance. This can be time-consuming and require some expertise in model building.

3. Convolution Variants

The goal of a CNN is to transform the input image into concise abstract representations of the original input. The individual convolutional layers try to find more complex patterns from the previous layer's observations. The logic is that 10 curved lines would form two ellipses, which would make an eye.

To do this, each layer uses a kernel, usually a 2x2 or 3x3 matrix, that slides through the previous layer's output to generate a new output. The word **convolve** from convolution means to roll or slide.

The variants of convolution operations are as follows:

3.1 Strided Convolution

A strided convolution is another basic building block of convolution that is used in Convolutional Neural Networks. Let's say we want to convolve this (7 * 7) image with this (3 * 3) filter, except, that instead of doing it the usual way, we're going to do it with a stride of (2).

$$\begin{array}{|c|c|c|c|c|c|c|} \hline 2 & 3 & 7 & 4 & 6 & 2 & 9 \\ \hline 6 & 6 & 9 & 8 & 7 & 4 & 3 \\ \hline 3 & 4 & 8 & 3 & 8 & 9 & 7 \\ \hline 7 & 8 & 3 & 6 & 6 & 3 & 4 \\ \hline 4 & 2 & 1 & 8 & 3^3 & 4^4 & 6^4 \\ \hline 3 & 2 & 4 & 1 & 9^1 & 8^0 & 3^2 \\ \hline 0 & 1 & 3 & 9 & 3^{-2} & 1^0 & 4^3 \\ \hline \end{array} \quad \begin{array}{|c|c|c|} \hline 3 & 4 & 4 \\ \hline 1 & 0 & 2 \\ \hline -1 & 0 & 3 \\ \hline \end{array} \quad = \quad \begin{array}{|c|c|c|} \hline 91 & 110 & 83 \\ \hline 69 & 91 & 127 \\ \hline 44 & 72 & 74 \\ \hline \end{array}$$

7×7 3×3 3×3

Convolutions with a stride of two

This means that we take the element-wise product as usual in this upper left (3 times 3) region, and then multiply and sum elements. That gives us (91). But then instead of stepping the blue box over by one step, we're going to step it over by two steps. It's illustrated how the upper left corner has gone from one dot to another jumping over one position. Then we do the usual element-wise product and summing, and that gives us (100). Next, we're going to do that again and make the blue box jump over by two steps. We obtain the value (83). Then, when we go to the next row, again we take two steps instead of one step. We will move filter by (2) steps and we'll obtain (69).

In this example we convolve (7 times 7) matrix with a (3 times 3) matrix and we get a (3 times 3) output. The input and output dimensions turns out to be governed by the following formula:

$$\frac{n - f + 2p}{s + 1}$$

If we have (n times n) image convolved with an (f times f) filter and if we use a padding (p) and a stride (s), in this example (s=2), then we end up with an output that is (n-f+2p). Because we're stepping (s) steps at the time instead of just one step at a time, we now divide by (s) and add (1). In our example, we have ((7-3+0)/2+1 = 4/2 +1 =3), that is why we end up with this (3 times 3) output. Notice that in this formula above, we round the value of this fraction, which generally might not be an integer value, down to the nearest integer.

3.2 Tiled Convolution

Tiled Convolutional Neural Networks are an extension to Convolutional Neural Networks that learn k separate convolution kernels within the same layer. These convolution operations are applied over every k 'th unit (hence the "tiling"). Even $k=2$ has been shown to give good results. The advantage of this is that through the pooling operation (where layers are "downsampled" by taking the max, average, or even stochastic combination of each $p \times p$ window in the output of a convolutional layer, across many tiles -- where $k=p$ has been shown to give good performance), the tiled layers can provide rotational and scale invariance as well as the translational invariance that comes from having convolutional layers in the first place.

Moreover, each convolution operation is effectively learning an additional feature (or map), which is a learned representation of the training data, and the tiled layers, like convolutional layers, also still have a relatively small number of learned parameters. In essence, it is the pooling operation over these multiple "tiled" maps that allows the network to learn invariances over scaling and rotation.

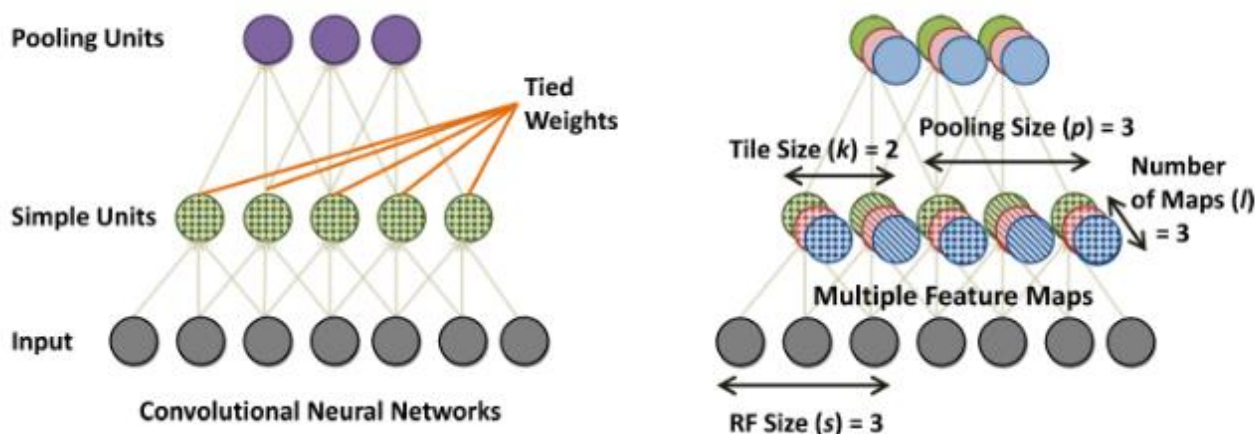


Fig. 8: CNN vs Tiled CNN

In above figure, units with the same color belong to the same map; within each map, units with the same fill texture have tied weights. We call this local untying of weights “tiling.” Tiled CNNs are parametrized by a tile size k : we constrain only units that are k steps away from each other to be tied. By varying k , we obtain a spectrum of models which trade off between being able to learn complex invariances, and having few learnable parameters. At one end of the spectrum we have traditional CNNs ($k = 1$), and at the other, we have fully untied simple units.

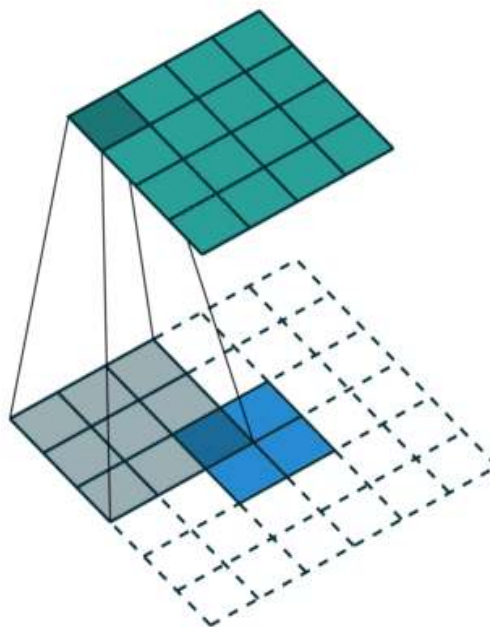
Next, we will allow our model to use multiple “maps,” so as to learn highly over complete representations. A map is a set of pooling units and simple units that collectively cover the entire image (see Figure 8 - Right). When varying the tiling size, we change the degree of weight tying within each map; for example, if $k = 1$, the simple units within each map will have the same weights. In our model, simple units in different maps are never tied. By having units in different maps learn different features, our model can learn a rich and diverse set of features. Tiled CNNs with multiple maps enjoy the twin benefits of (i) being able to represent complex invariances, by pooling over (partially) untied weights, and (ii) having a relatively small number of learnable parameters.

3.3 Transposed Convolution

The transposed convolutional layer, unlike the convolutional layer, is up sampling in nature. Transposed convolutions are usually used in auto-encoders and GANs, or generally any network that must reconstruct an image.

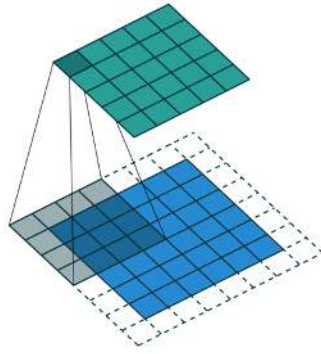
The word **transpose** means to cause two or more things to switch places with each other, and in the context of convolutional neural networks, this causes the input and the output dimensions to switch.

In a transposed convolution, instead of the input being larger than the output, the output is larger. An easy way to think of it is to picture the input being padded until the corner kernel can just barely reach the corner of the input.

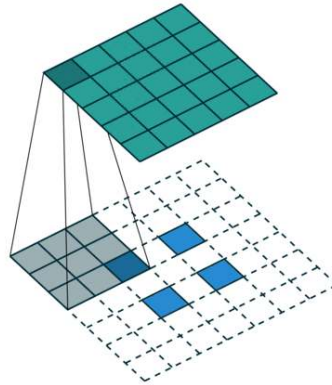


When down sampling and up sampling techniques are applied to transposed convolutional layers, their effects are reversed. The reason for this is for a network to be able to use convolutional layers to compress the image, then transposed convolutional layers with the exact same down sampling and up sampling techniques to reconstruct the image.

When padding is ‘added’ to the transposed convolutional layer, it **seems** as if padding is removed from the input, and the resulting output becomes smaller.



Without padding, the output is 7x7, but with padding on both sides, it is 5x5. When strides are used, they instead affect the input, instead of the output.



Strides (2, 2) increases the output dimension from 3x3 to 5x5.

• Transposed Convolution vs Deconvolution

Deconvolution is a term floating around next to transposed convolutions, and the two are often confused for each other. Many sources use the two interchangeably, and while deconvolutions do exist, they are not very popular in the field of machine learning.

A deconvolution is a mathematical operation that reverses the effect of convolution. Imagine throwing an input through a convolutional layer, and collecting the output. Now throw the output through the deconvolutional layer, and you get back the exact same input. It is the inverse of the multivariate convolutional function.

On the other hand, a transposed convolutional layer only reconstructs the spatial dimensions of the input. In theory, this is fine in deep learning, as it can learn its own parameters through gradient descent, however, it does not give the same output as the input.

3.4 Dilated Convolution

Dilated convolution, also known as atrous convolution, is a type of convolution operation used in convolutional neural networks (CNNs) that enables the network to have a larger receptive field without increasing the number of parameters. It is a technique that expands the kernel (input) by inserting holes between its consecutive elements. In simpler terms, it is the same as convolution but it involves pixel skipping, so as to cover a larger area of the input.

In a regular convolution operation, a filter of a fixed size slides over the input feature map, and the values in the filter are multiplied with the corresponding values in the input feature map to produce a single output value. The receptive field of a neuron in the output feature map is defined as the area in the input feature map that the filter can “see”. The size of the receptive field is determined by the size of the filter and the stride of the convolution.

In contrast, in a dilated convolution operation, the filter is “dilated” by inserting gaps between the filter values. The dilation rate determines the size of the gaps, and it is a hyperparameter that can be adjusted. When the dilation rate is 1, the dilated convolution reduces to a regular convolution.

The dilation rate effectively increases the receptive field of the filter without increasing the number of parameters, because the filter is still the same size, but with gaps between the values. This can be useful in situations where a larger receptive field is needed, but increasing the size of the filter would lead to an increase in the number of parameters and computational complexity.

Dilated convolutions have been used successfully in various applications, such as semantic segmentation, where a larger context is needed to classify each pixel, and audio processing, where the network needs to learn patterns with longer time dependencies.

An additional parameter l (dilation factor) tells how much the input is expanded. In other words, based on the value of this parameter, $(l-1)$ pixels are skipped in the kernel. **Figure 9** depicts the difference between normal vs dilated convolution. In essence, normal convolution is just a **1-dilated convolution**.

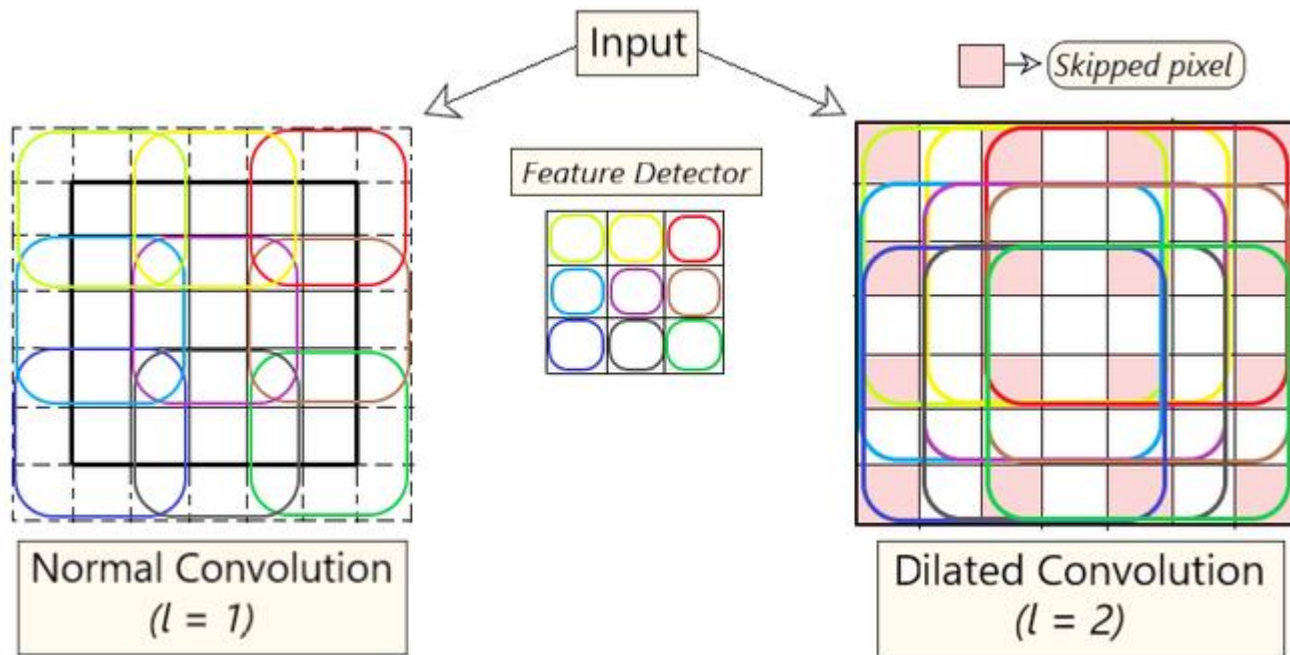


Fig 9: Normal Convolution vs Dilated Convolution

• Intuition:

Dilated convolution helps expand the area of the input image covered without pooling. The objective is to cover more information from the output obtained with every convolution operation. This method offers a wider field of view at the same computational cost. We determine the value of the dilation factor (l) by seeing how much information is obtained with each convolution on varying values of l .

By using this method, we are able to obtain more information without increasing the number of kernel parameters. In **Fig 9**, the image on the left depicts dilated convolution. On keeping the value of $l = 2$, we skip 1 pixel ($l - 1$ pixel) while mapping the filter onto the input, thus covering more information in each step.

• Formula Involved:

$$(F_{*lk})(p) = \sum_{(s+lt=p)} F(s)k(t)$$

where,

$F(s)$ = Input

$k(t)$ = Applied Filter

$*l$ = l -dilated convolution

$(F_{*lk})(p)$ = Output

• Advantages of Dilated Convolution:

Using this method rather than normal convolution is better as:

1. **Larger receptive field** (i.e. no loss of coverage)
2. **Computationally efficient** (as it provides a larger coverage on the same computation cost)

3. *Lesser Memory consumption (as it skips the pooling step) implementation*
4. *No loss of resolution of the output image (as we dilate instead of performing pooling)*
5. *Structure of this convolution helps in maintaining the order of the data.*

• **Disadvantages of dilated convolutions are:**

1. Reduced spatial resolution in the output feature map compared to the input feature map
2. Increased computational cost compared to regular convolutions with the same filter size and stride

4. CNN Learning

A neural network without an activation function is essentially just a linear regression model. The activation function does the non-linear transformation to the input making it capable to learn and perform more complex tasks.

4.1 Non Linearity Functions

Nonlinear functions play a crucial role in Convolutional Neural Networks (CNNs) by introducing complex transformations that allow the network to capture intricate patterns and relationships in the data. In CNNs, these nonlinear functions are typically applied after convolutional and pooling layers to introduce nonlinearity into the network architecture. The most commonly used nonlinear function in CNNs is the Rectified Linear Unit (ReLU), but there are other options as well. Here are some common nonlinear activation functions used in CNNs:

- **Rectified Linear Unit (ReLU):** The ReLU activation function is defined as $f(x) = \max(0, x)$. It replaces all negative values with zero and keeps positive values unchanged. ReLU is computationally efficient and helps mitigate the vanishing gradient problem, allowing deeper networks to be trained effectively.
- **Leaky ReLU:** The Leaky ReLU is an extension of the ReLU function that allows a small gradient for negative values to prevent neurons from becoming inactive. It's defined as $f(x) = x$ if $x > 0$, and $f(x) = \alpha x$ if $x < 0$, where α is a small positive constant.
- **Parametric ReLU (PReLU):** PReLU is similar to Leaky ReLU, but the slope for negative values is learned during training rather than being a fixed constant. This can lead to improved performance, especially on large datasets.
- **Exponential Linear Unit (ELU):** The ELU activation function is defined as $f(x) = x$ for $x > 0$, and $f(x) = \alpha * (\exp(x) - 1)$ for $x < 0$, where α is a positive constant. ELU can help alleviate the vanishing gradient problem and produce smoother gradients.
- **Scaled Exponential Linear Unit (SELU):** SELU is a variant of ELU that aims to maintain mean and variance stability in neural networks. It's designed to automatically adjust its parameters to achieve this stability, making it particularly useful in deeper architectures.
- **Hyperbolic Tangent (tanh):** The tanh activation function squashes values to the range of -1 to 1. It is symmetric around the origin, so it can produce both positive and negative values.
- **Sigmoid:** The sigmoid activation function maps inputs to values between 0 and 1. It's often used in the output layer for binary classification problems where the output represents a probability.
- **Swish:** Swish is a recently introduced activation function that combines elements of ReLU and sigmoid functions. It's defined as $f(x) = x * \text{sigmoid}(\beta x)$, where β is a learnable parameter. Swish has shown promising performance in certain cases. The choice of activation function depends on the specific problem, architecture, and dataset. It's common practice to experiment with different activation functions to find the one that works best for a given task.

Here we will look into the ReLU activation function, more specifically about it's non - linear behaviour. ReLU is an acronym for Rectified Linear Unit. It is the most commonly used activation function. The function returns 0 if it receives any negative input, but for any positive value x it returns that value back. So, Mathematically it can be expressed as:- $f(x) = \max(0, x)$ Basically, it sets anything less than or equal to 0 (negative numbers) to be 0. And keeps all the same values for any values > 0 . Graphical representation of ReLU function is:

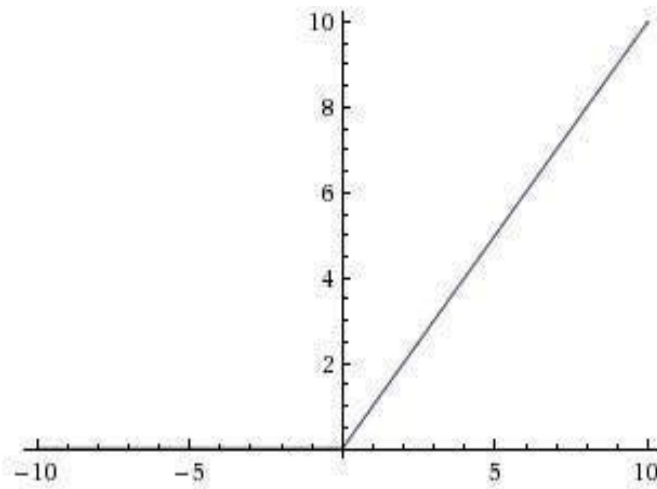


Fig. 10 RELU Activation Function

From the graphical representation, we observe that it is a very simple function. This means, it is composed of two pieces of straight lines only which are separated by y-axis of the graph. Also it includes very simple mathematical operations that are why it is less computationally expensive than other activation functions. Derivative of ReLU function By just looking into the equation of ReLU function it's not clear that what the derivative will be, However let's look into the graph so that it may get clear to me about it's derivatives. Let's draw a graph of ReLU function where x is ranging from -4 to +4, and increment by 1 unit. Similarly y axis is labelled as $f(x)$, value of function at x .

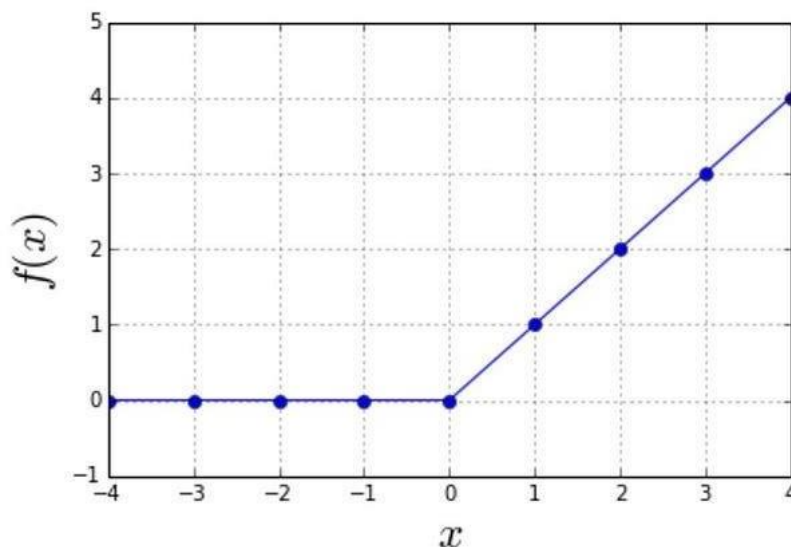


Fig.11 RELU function

Now we will look into the derivative of the ReLU using above graph. So, let us see the derivative at different values of x . For example let's see the derivative for both positive value of x and negative value of x .

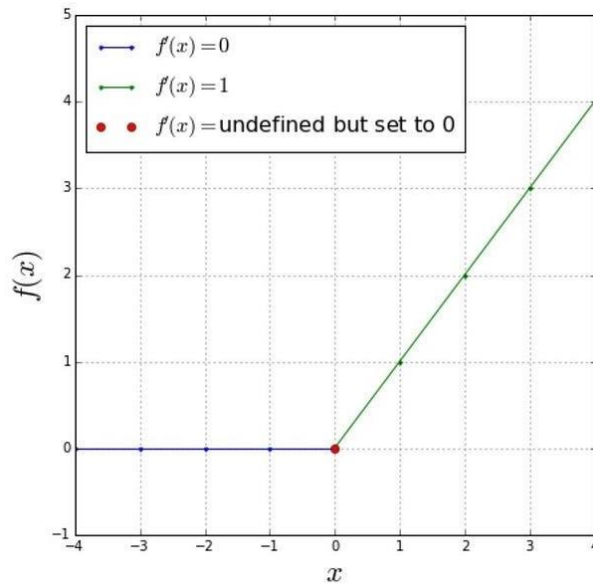


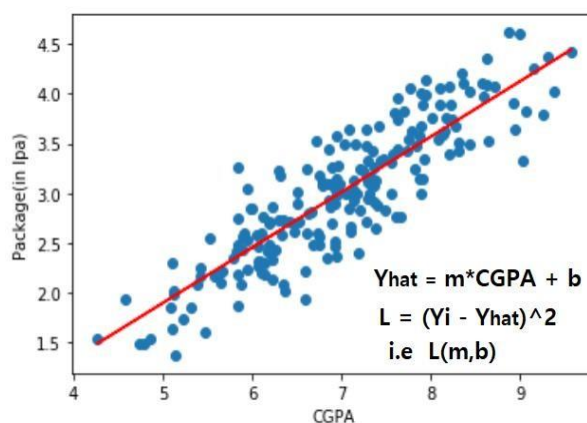
Fig. 12 Derivative function of ReLU

As we know the derivative of function is defined as the slope of the function at certain point. So you can see that the function is mostly differentiable. If x is greater than 0 the derivative is 1 and if x is less than zero the derivative is 0. But when $x = 0$, the derivative does not exist. There are two ways to deal with this. First, you can just arbitrarily assign a value for the derivative of $y = f(x)$ when $x = 0$. A second alternative is, instead of using the actual $y = f(x)$ function, use an approximation to ReLU which is differentiable for all values of x . Anyway, Till now we were getting confused that actually what the ReLU is Linear or Nonlinear? We know that Mathematically, it is clear that, A function is linear if the slope is constant in its complete domain and the ReLU function is non-differentiable around 0, but the slope is always either 0 (for negative values) or 1 (for positive values). That's why the ReLU function is Non-Linear. Intuitively, we can understand that as The ReLU is an activation function and the purpose of activation function is to introduce non-linearity in the neural network.

4.2 Loss Functions

The loss function is very important in machine learning or deep learning. In mathematical optimization and decision theory, a loss or cost function (sometimes also called an error function) is a function that maps an event or values of one or more variables onto a real number intuitively representing some “cost” associated with the event. In simple terms, the Loss function is a method of evaluating how well your algorithm is modeling your dataset. It is a mathematical function of the parameters of the machine learning algorithm.

In simple linear regression, prediction is calculated using slope(m) and intercept(b). The loss function for this is the $(Y_i - \hat{Y}_i)^2$ i.e loss function is the function of slope and intercept.



Figur

Cost Function vs Loss Function

Loss Function	Cost Function
Measures the error between predicted and actual values in a machine learning model	Quantifies the overall cost or error of the model on the entire training set
Used to optimize the model during training	Used to guide the optimization process by minimizing the cost or error
Can be specific to individual samples	Aggregates the loss values over the entire training set
Examples include mean squared error (MSE), mean absolute error (MAE), and binary cross-entropy.	Often the average or sum of individual loss values in the training set
Used to evaluate model performance	Used to determine the direction and magnitude of parameter updates during optimization
Different loss functions can be used for different tasks or problem domains	Typically derived from the loss function, but can include additional regularization terms or other considerations

Loss Function in Deep Learning

- Regression
 - MSE(Mean Squared Error)
 - MAE(Mean Absolute Error)
 - Hubber loss
- Classification
 - Binary cross-entropy
 - Categorical cross-entropy

A. Regression Loss

1. Mean Squared Error / Squared loss / L2 loss

The Mean Squared Error (MSE) is the simplest and most common loss function. To calculate the MSE, you take the difference between the actual value and model prediction, square it, and average it across the whole dataset.

$$MSE = \frac{1}{N} \sum_i^N (Y_i - \hat{Y}_i)^2$$

- Advantage
 - Easy to interpret
 - Always differential because of the square

- Only one local minima
- **Disadvantage**
 - Error unit in the square. Because the unit in the square is not understood properly
 - Not robust to outlier

Note—In regression at the last neuron use linear activation function.

2. Mean Absolute Error / L1 loss

The Mean Absolute Error (MAE) is also the simplest loss function. To calculate the MAE, you take the difference between the actual value and model prediction and average it across the whole dataset.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |Y_i - \hat{Y}_i|$$

- **Advantage**
 - Intuitive and easy
 - Error Unit Same as the output column
 - Robust to outlier
- **Disadvantage**
 - Graph, not differential. We cannot use gradient descent directly, then we can sub gradient calculation.

Note—In regression at the last neuron use linear activation function.

3. Huber Loss

In statistics, the Huber loss is a loss function used in robust regression, that is less sensitive to outliers in data.

$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 \quad |y_i - \hat{y}_i| \leq \delta$$

$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) \quad |y_i - \hat{y}_i| > \delta$$

n—the number of data points

y—the actual value of the data point. Also known as true value

$\hat{y}$ —the predicted value of the data point. This value is returned by the model

δ —defines the point where the Huber loss function transitions from a quadratic to linear

- **Advantage**
 - Robust to outlier
 - It lies between MAE and MSE

- **Disadvantage**

- Its main disadvantage is the associated complexity. In order to maximize model accuracy, the hyperparameter δ will also need to be optimized which increases the training requirements.

B. Classification Loss

1. Binary Cross Entropy / log loss

It is used in binary classification problems like two classes. Example a person has covid or not or my article gets popular or not.

Binary cross entropy compares each of the predicted probabilities to the actual class output which can be either 0 or 1. It then calculates the score that penalizes the probabilities based on the distance from the expected value. That means how close or far from the actual value than the squared error loss.

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)$$

Y_i – actual values

y_{ihat} – neural network prediction

- **Advantage**

- A cost function is a differential

- **Disadvantage**

- Multiple local minima
- Not intuitive

Note–In classification at last neuron use sigmoid activation function.

2. Categorical Cross Entropy

Categorical Cross entropy is used for Multiclass classification and softmax regression.

$$\text{Loss} = -\sum_{j=1}^K y_j \log(\hat{y}_j)$$

where k is number of classes in the data

Where k is classes

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k [y_{ij} \log(\hat{y}_{ij})]$$

Where,

k is classes,

y-actual value

yhat–Neural Network prediction

Note – In multi-class classification at the last neuron use the softmax activation function.

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

If problem statement have 3 classes

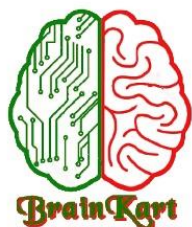
softmax activation $f(z) = e^{z_1} / (e^{z_1} + e^{z_2} + e^{z_3})$

If target column has One hot encode to classes like 0 0 1, 0 1 0, 1 0 0 then use categorical cross-entropy. And if the target column has numerical encoding to classes like 1,2,3,4....n then use sparse categorical cross-entropy. Sparse categorical cross-entropy faster than categorical cross-entropy.

4.3 Gradient Computation

Gradient computation is a fundamental concept in deep learning and optimization. It involves calculating the gradient of a mathematical function with respect to its input variables. In the context of deep learning, the function typically represents a loss or cost function that quantifies how well a model's predictions match the actual target values. The gradient of a function provides information about its local rate of change. In other words, it tells you how the function's output changes as you make small adjustments to its input variables. This information is crucial for optimization algorithms, which aim to minimize the value of the loss function in order to train a machine learning model effectively. In deep learning, gradient computation is used primarily for two purposes:

1. **Backpropagation:** Backpropagation is a key algorithm for training neural networks. It involves computing the gradients of the loss function with respect to the model's parameters (weights and biases) for each layer in the network. These gradients indicate how much each parameter needs to be adjusted to minimize the loss. Backpropagation relies on the chain rule of calculus to efficiently calculate these gradients layer by layer
2. **Gradient Descent:** Gradient descent is an optimization algorithm that uses the gradients of the loss function to iteratively update the model's parameters in a way that reduces the loss. The basic idea is to take steps in the opposite direction of the gradient to reach a local minimum of the loss function. This process is repeated until the algorithm converges to a set of parameter values that hopefully result in a well-trained model



www.BrainKart.com

Anna University

for Affiliated Engineering College - 2021 Regulation



AID (Artificial Intelligence & Data Science Engineering)

1st Semester ▶

2nd Semester ▶

3rd Semester ▶

4th Semester ▶

5th Semester ▶

6th Semester ▶

7th Semester ▶

8th Semester ▶

Click on Subject/Paper under Semester to enter.

1st Semester

[Professional English - I - HS3152](#)

[Matrices and Calculus - MA3151](#)

[Engineering Physics - PH3151](#)

[Engineering Chemistry - CY3151](#)

[Problem Solving and Python Programming - GE3151](#)

2nd Semester

[Professional English - II - HS3252](#)

[Statistics and Numerical Methods - MA3251](#)

[Engineering Graphics - GE3251](#)

[Physics for Information Science - PH3256](#)

[Basic Electrical and Electronics Engineering - BE3251](#)

[Data Structures Design - AD3251](#)

3rd Semester

[Discrete Mathematics - MA3354](#)

[Digital Principles and Computer Organization - CS3351](#)

[Database Design and Management - AD3391](#)

[Design and Analysis of Algorithms - AD3351](#)

[Data Exploration and Visualization - AD3301](#)

[Artificial Intelligence - AL3391](#)

4th Semester

[Environmental Sciences and Sustainability - GE3451](#)

[Probability and Statistics - MA3391](#)

[Operating Systems - AL3452](#)

[Machine Learning - AL3451](#)

[Fundamentals of Data Science and Analytics - AD3491](#)

[Computer Networks - CS3591](#)

5th Semester

[Deep Learning - AD3501](#)

[Data and Information Security - CW3551](#)

[Distributed Computing - CS3551](#)

[Big Data Analytics - CCS334](#)

[Elective 1](#)

[Elective 2](#)

6th Semester

[Embedded Systems and IoT - CS3691](#)

[Open Elective-1](#)

[Elective-3](#)

[Elective-4](#)

[Elective-5](#)

[Elective-6](#)

7th Semester

[Human Values and Ethics - GE3791](#)

[Open Elective 2](#)

[Open Elective 3](#)

[Open Elective 4](#)

[Management Elective](#)

8th Semester

[Project Work / Internship](#)



Anna University Notes

Therithal Info
Contains ads

3.7★

199 reviews

50K+

Downloads

3+

Rated for 3+ ©

Install



BrainKart: Learning, Study App

Therithal Info
Contains ads

4.5★

160 reviews

10K+

Downloads

3+

Rated for 3+ ©

Install

All Computer Engg Subjects - [B.E., M.E.,]

(Click on Subjects to enter)

<u>Programming in C</u>	<u>Computer Networks</u>	<u>Operating Systems</u>
<u>Programming and Data Structures I</u>	<u>Programming and Data Structure II</u>	<u>Problem Solving and Python Programming</u>
<u>Database Management Systems</u>	<u>Computer Architecture</u>	<u>Analog and Digital Communication</u>
<u>Design and Analysis of Algorithms</u>	<u>Microprocessors and Microcontrollers</u>	<u>Object Oriented Analysis and Design</u>
<u>Software Engineering</u>	<u>Discrete Mathematics</u>	<u>Internet Programming</u>
<u>Theory of Computation</u>	<u>Computer Graphics</u>	<u>Distributed Systems</u>
<u>Mobile Computing</u>	<u>Compiler Design</u>	<u>Digital Signal Processing</u>
<u>Artificial Intelligence</u>	<u>Software Testing</u>	<u>Grid and Cloud Computing</u>
<u>Data Ware Housing and Data Mining</u>	<u>Cryptography and Network Security</u>	<u>Resource Management Techniques</u>
<u>Service Oriented Architecture</u>	<u>Embedded and Real Time Systems</u>	<u>Multi - Core Architectures and Programming</u>
<u>Probability and Queueing Theory</u>	<u>Physics for Information Science</u>	<u>Transforms and Partial Differential Equations</u>
<u>Technical English</u>	<u>Engineering Physics</u>	<u>Engineering Chemistry</u>
<u>Engineering Graphics</u>	<u>Total Quality Management</u>	<u>Professional Ethics in Engineering</u>
<u>Basic Electrical and Electronics and Measurement Engineering</u>	<u>Problem Solving and Python Programming</u>	<u>Environmental Science and Engineering</u>

